SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Performance prediction of iterator based database systems with decision trees

Hoang-Nam Tran

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Performance prediction of iterator based database systems with decision trees

Performancevorhersage iteratorbasierter Datenbanksysteme mit Entscheidungsbäumen

Author:	Hoang-Nam Tran
Examiner:	Prof. Dr. Thomas Neumann
Supervisor:	Maximilian Rieger, M.Sc.
Submission Date:	27.04.2025

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 27.04.2025 Hoang-Nam Tran

Acknowledgments

Abstract

Contents

Acknowledgments **iv**

Abstract **v**

1 Introduction **1**
1.1 Section . . . 1
1.1.1 Subsection . . . 1

2 Theoretical Background **3**
2.1 Databases and Query Execution . . . 3
2.1.1 Iterator-Based Execution (PostgreSQL) . . . 4
2.1.2 Compiled Query Execution (Umbra) . . . 5
2.1.3 Pipelines and Pipeline Breakers . . . 6
2.2 Database Performance Prediction . . . 7
2.2.1 Benchmarking . . . 7
2.2.2 Error Metrics . . . 7
2.2.3 Traditional Cost Models and Learned Approaches . . . 7
2.2.4 Neural Networks, Decision Trees, and T3 . . . 7
2.3 Related Work . . . 7
2.4 ——————— . . . 7
2.5 Database Basics . . . 7
2.6 Performance Prediction . . . 7
2.7 Benchmarking . . . 8
2.8 Cost Models . . . 8
2.9 Decision Trees . . . 8
2.10 PostgreSQL . . . 8
2.11 Umbra . . . 8
2.12 T3 . . . 8
2.13 Section . . . 8
2.13.1 Subsection . . . 8

3 Methodology **10**
3.1 Problem Statement and Prediction Target . . . 10

3.2 Data and Preprocessing 10
3.3 Implementation 11
3.3.1 Mapping Parsed PostgreSQL Plans to the T3 Plan Format 11
3.3.2 Pipeline Decomposition 13
3.3.3 PostgreSQL-Native Feature Vectors 13
3.4 Training 15
3.4.1 Training Targets and Model Configuration 17
3.4.2 Data Splits and Reproducibility 17
3.5 Other Performance Prediction Implementations 17
3.6 Section 18
3.6.1 Subsection 18

4 Evaluation **20**
4.1 Ablation Study 20
4.1.1 PostgreSQL-native Features vs. Direct Transfer 20
4.1.2 Actual vs. Estimated Cardinalities 20
4.1.3 Pipeline vs Tuple targets 20
4.2 Section 20
4.2.1 Subsection 20

5 Future Work **22**
5.1 Section 22
5.1.1 Subsection 22

6 Conclusion **24**
6.1 Section 24
6.1.1 Subsection 24

Abbreviations **26**

List of Figures **27**

List of Tables **28**

1 Introduction

1.1 Section

Citation test [**latex**].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}`, `\ac{TUM}` ⇒ Technical University of Munich (TUM), TUM

For more details, see the documentation of the `acronym` package[1].

1.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 1.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

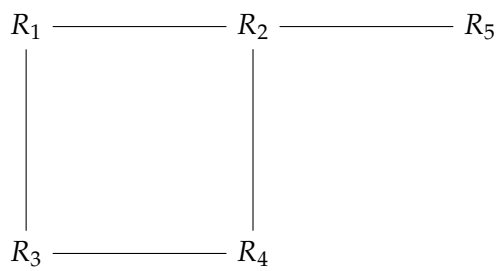

Figure 1.1: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

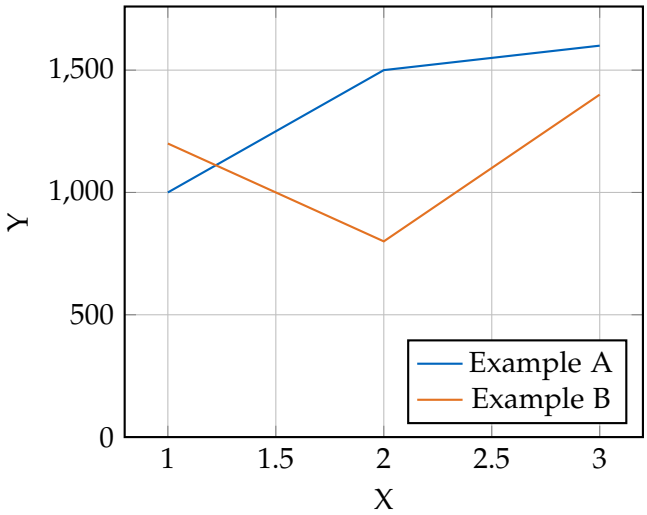

Figure 1.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 1.3: An example for a source code listing.

2 Theoretical Background

2.1 Databases and Query Execution

A *database management system* (DBMS) is software that stores, organizes, and retrieves structured data. Users interact with such systems primarily through *Structured Query Language* (SQL), a declarative language that lets users specify what data they want rather than how to retrieve it [**silberschatz2019**]. For example, a simple SQL query might select all orders placed by a particular customer by joining an `orders` table with a `customers` table and filtering by a customer name. Because SQL describes the desired result and not the retrieval procedure, the DBMS is free to choose any execution strategy that produces the correct answer.

When a DBMS receives an SQL query, it processes the query in several successive phases [**silberschatz2019**]:

1. **Parsing.** The SQL text is checked for syntactic correctness and translated into an internal *parse tree* that represents the query's structure.

2. **Rewriting.** The parse tree is rewritten to apply logical transformations, such as resolving *view* definitions. A view is a named, stored query that acts as a virtual table and is expanded into its underlying query during this phase.

3. **Optimization.** A query *optimizer* considers many possible ways to execute the query — for instance, different join orderings or different index strategies.

   Here, it considers the *cardinality* of each *operator*, which is the number of rows (tuples) that the operator is expected to produce or consume. An operator is a single processing step in the query, such as a table scan, a join, or a sort. Since the query has not been executed yet, the optimizer works with *estimated cardinalities*, derived from statistical summaries of the data such as table sizes and column value distributions. These estimates directly influence which plan the optimizer considers cheapest.

   Based on its cost estimates, the optimizer selects the plan it believes to be cheapest.

   The *actual cardinality* - the true number of rows processed - only becomes known after execution. Estimated and actual cardinalities can differ, which is a recurring challenge in performance prediction.

4. **Plan generation.** The result of optimization is a *query plan*: a tree of operators such as table scans, joins, sorts, and aggregations. This tree encodes exactly how the query will be executed and in what order.

5. **Execution.** The query plan is handed to the execution engine, which processes it and produces the query result.

The query plan is central to performance, since the optimizer's choice of operators and their ordering directly determines how much work the DBMS has to perform. Different DBMSs implement these phases differently, and especially the execution step varies widely. *PostgreSQL*, for instance, executes plans using an iterator-based model described below. *Umbra*, by contrast, compiles query plans into native machine code at runtime. These architectural choices have a large impact on query execution time [**neumann2011**].

2.1.1 Iterator-Based Execution (PostgreSQL)

PostgreSQL (PG) uses the *iterator model* for query execution, which is also known as the *Volcano model* or *pull-based execution* after the system that popularized it [**graefe1994**].

In this model, every operator in the query plan tree exposes a uniform three-call interface: `open()`, `next()`, and `close()`. The central call is `next()`: when invoked, the operator produces and returns the next output tuple (i.e., one row of data). If no more tuples are available, the operator signals end-of-stream. The `open()` call initializes the operator and its state, and `close()` releases any resources held.

Operators are composed into a tree that mirrors the query plan. The root operator drives execution: each time it needs another tuple, it calls `next()` on its child operator. That child, in turn, calls `next()` on its own children, and so on, all the way down to the leaf operators — typically table scans — which actually read data from disk or memory. Tuples thus *flow upward* through the tree one at a time, on demand. This is the defining property of pull-based execution: no operator pushes data forward; instead, every operator waits to be asked.

As a concrete example, consider a query that scans a table, filters rows by a predicate, and then sorts the result. The tree would consist of a sort operator at the root, a filter operator as its child, and a sequential scan at the leaf. When execution starts, the sort operator repeatedly calls `next()` on the filter, which in turn calls `next()` on the scan, until all matching rows have been collected. Only then does the sort operator produce its first output tuple.

PG exposes this plan tree through the `EXPLAIN` and `EXPLAIN ANALYZE` commands. `EXPLAIN` shows the chosen plan with the optimizer's cost and cardinality estimates for each node. `EXPLAIN ANALYZE` additionally executes the query and annotates every node

with the actual number of rows produced and the actual time spent [**postgresql_docs_query_path**]. This makes `EXPLAIN ANALYZE` output the primary source of per-operator timing information in PG and is also the format used by the Zero-Shot benchmark that this thesis builds on [**zeroshot**].

A key advantage of the iterator model is its simplicity and composability. New operators can be added independently, as long as they implement the three-call interface. However, this design also has a well-known downside: calling `next()` individually for every single tuple introduces per-tuple function call overhead and hurts CPU branch prediction and cache behavior [**graefe1994**]. This is one reason why more recent systems like Umbra take a different approach and compile query plans to native code, as described in the next subsection.

2.1.2 Compiled Query Execution (Umbra)

Umbra is a relational DBMS developed at the Technical University of Munich that is designed for high performance on modern hardware [**umbra2020**]. Unlike PG's interpreter-based approach, Umbra takes a fundamentally different execution strategy: it compiles each query plan into native machine code at query time, using the LLVM compiler framework [**neumann2011**].

The motivation for this is to eliminate the per-tuple overhead of the iterator model. Recall that in PG's pull-based execution, every single tuple causes a chain of `next()` calls to travel through the operator tree. This introduces substantial function call overhead and makes it difficult for the CPU to predict branches and keep data in cache. In a compiled system, these operator boundaries dissolve. Instead of calling functions for each tuple, Umbra generates a single piece of native machine code that fuses the logic of multiple operators into tight loops. Data is *pushed* forward through the code rather than pulled upward through function calls.

Concretely, the compilation model works as follows [**neumann2011**]. Each operator in the query plan implements two code-generation methods: `produce()` and `consume()`. When `produce()` is called on an operator, it generates code that drives data production, typically by calling `produce()` on its child. When a child has produced a tuple, it calls `consume()` on its parent, which generates the code to process that tuple and pass it upward. The result is a chain of nested loops in the generated code, where data flows from the scan at the bottom all the way to the top operator without any function call overhead at operator boundaries. This generated intermediate representation is then handed to LLVM, which applies its own compiler optimizations and produces executable native machine code. The compilation step itself takes some time — typically milliseconds — but for queries that run for tens of milliseconds or longer, this overhead is easily amortized.

As a concrete consequence for this thesis: T3, the performance prediction model this work builds on, was originally designed for Umbra and therefore relies on Umbra's query plans and their structure [**t3**]. The key structural concept that Umbra's compilation naturally exposes is the notion of *pipelines*, which is described next.

2.1.3 Pipelines and Pipeline Breakers

A central concept in compiled query execution — and in T3's prediction approach — is the decomposition of a query plan into *pipelines* [**t3**].

A *pipeline* is a maximal sequence of operators that can process tuples in a fully streaming fashion, without any need to fully materialize an intermediate result. Within a pipeline, each tuple produced by the scan operator at the bottom of the pipeline flows immediately through all subsequent operators — being filtered, transformed, or joined — until it reaches the end of the pipeline. No tuple has to wait for any other tuple to be processed. In compiled execution, an entire pipeline compiles down to a single tight loop: one tuple enters at the scan, travels through all operators in the pipeline, and either reaches the next pipeline boundary or leaves the query entirely.

A *pipeline breaker* is an operator that interrupts this streaming flow, because it must see *all* of its input before it can produce any output. The following are typical examples of pipeline breakers [**t3**]:

- **Sort.** A sort operator cannot emit a single sorted tuple until it has collected all tuples and sorted them.
- **Aggregation (build phase).** A group-by operator must first accumulate all input tuples into groups before it can output any aggregated result.
- **Hash join (build side).** The inner (build) side of a hash join must be fully read into a hash table before any probing can begin. Consequently, the build side forms its own pipeline, ending at the hash table.
- **Materialize.** An explicit materialization operator stores all input tuples before they can be scanned again by a downstream operator.

Not every operator is a pipeline breaker. A filter, a projection, or the probe side of a hash join — where tuples from the outer relation are looked up in the already-built hash table — do not require full materialization and can therefore remain inside a pipeline. In particular, a single query pipeline can contain multiple consecutive join probe stages.

A query plan thus decomposes into a directed acyclic graph (DAG) of pipelines. Each pipeline starts either at a base table scan or at the scan output of a pipeline breaker,

processes tuples in a streaming fashion, and ends either at a pipeline breaker or at the final output. Figure 3.1 provides an example of a plan with such a structure from the benchmark data used in this thesis.

This decomposition is fundamental to T3's performance prediction approach [**t3**]. Rather than predicting the runtime of a whole query in a single step, T3 predicts the runtime of each pipeline individually and sums the per-pipeline predictions to obtain the total query runtime. Intuitively, each pipeline represents a distinct computational unit with its own characteristic workload — its scan cardinality, the operators it contains, and the fraction of tuples that reach each operator. Predicting per-pipeline is therefore simpler and more accurate than predicting for the whole query at once. The pipelines and their structure are also the basis for the feature vectors used by T3, as discussed in the Methodology chapter.

2.2 Database Performance Prediction

2.2.1 Benchmarking

sowas auch schreiben wie ssb, tpch, etc.

2.2.2 Error Metrics

2.2.3 Traditional Cost Models and Learned Approaches

2.2.4 Neural Networks, Decision Trees, and T3

2.3 Related Work

2.4 ——————————

2.5 Database Basics

operator type, estimated/actual cardinalities, widths, and runtimes

2.6 Performance Prediction

draft: beschreiben, was performance prediction ist, was die ziele sind, etc. benchmarking, etc. Cost Model beschreiben

2.7 Benchmarking

timed queries, etc.

2.8 Cost Models

2.9 Decision Trees

2.10 PostgreSQL

2.11 Umbra

2.12 T3

2.13 Section

Citation test [**latex**].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

2.13.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 2.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

[1] `https://ctan.org/pkg/acronym`

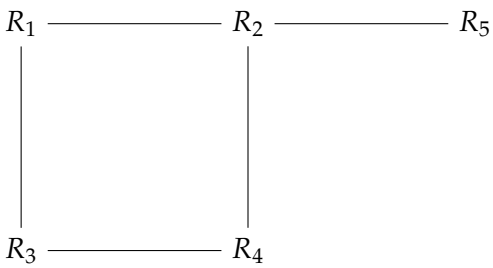

Figure 2.1: An example for a simple drawing.

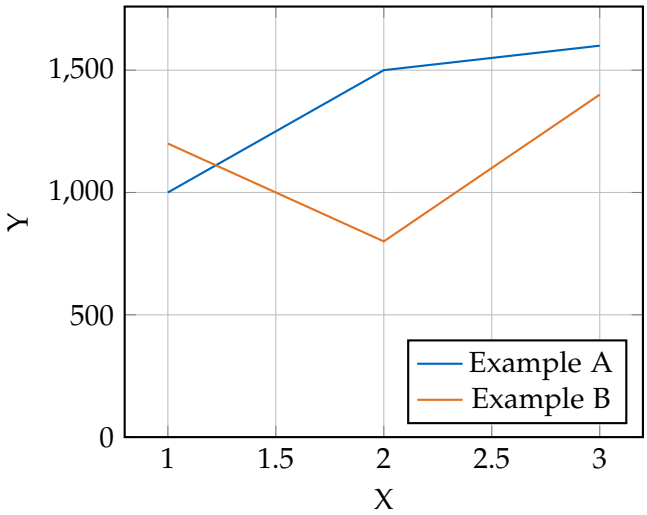

Figure 2.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 2.3: An example for a source code listing.

3 Methodology

3.1 Problem Statement and Prediction Target

In this thesis, we address the transfer of the T3 runtime prediction approach from Umbra to PostgreSQL parsed plans. The central methodological goal is to preserve T3's core modeling principles. Upon that, we adapt plan conversion and feature extraction to PostgreSQL-specific plan information.

The final prediction target is total query runtime. Each query plan is decomposed into pipelines and the model predicts a duration for each pipeline. The predicted query runtime is the sum of these per-pipeline predictions.

3.2 Data and Preprocessing

Regarding the training data, we use the public Zero-Shot benchmark data introduced with the Zero-Shot Cost Model. The benchmark is particularly suitable for this thesis because it consists of real-world datasets (e.g., IMDB) and established benchmark workloads (e.g., SSB, TPC-H and JOB). Overall, the benchmark contains 20 databases with large variation in tables, columns, and foreign-key relationships [**zeroshot**].

Having this multitude of databases allows us to test the generalization capabilities of our model. Similar to the T3 paper, we can train the model on all but one databases and test on the remaining one. Moreover, as the widely used JOB workload is included, we can compare our performance predictions to other implementations later on. Further, PostgreSQL is the intended database system of our implementation, which avoids the effort of building a separate PostgreSQL benchmark.

Following the benchmark construction described in the paper, queries are generated per database and executed on a PostgreSQL DBMS (v12) on c8220 nodes on the CloudLab platform. Then, they are stored as raw traces (i. e., the output of `EXPLAIN ANALYZE`). Next, the authors preprocess the queries to remove very long-running queries (queries above 30 s are excluded) to keep the benchmark practical and reproducible [**zeroshot**]. A further preprocessing step is to parse the raw traces into structured plan representations. These `parsed_plans` (15000 per database) contain the operator nodes found in the raw traces, but organize them in a tree structure with node-wise

attributes. For every node, e.g., operator type, estimated/actual cardinalities, widths, and runtimes are stored in the `plan_parameters` field.

Figure 3.1 shows such a structured plan representation for a `Hash Join` and its children. Here, also filter predicates can be retrieved from the `filter_columns` field. Furthermore, PostgreSQL-specific fields like `workers_planned` (planned parallel workers for the node) are available.

The tree structure can be used to create pipelines and the node information will be used to train the model. In the upcoming sections, we will describe these key implementation details.

Consequently, the parsed plans offer a good starting point for our pipeline and we can begin without further preprocessing.

3.3 Implementation

This section describes how a single parsed plan from the Zero-Shot benchmark is turned into a T3-style representation including pipelines. Then, we derive fixed-length feature vectors for each pipeline.

The procedure follows the pipeline prediction idea of T3 [**t3**], but also replaces Umbra-specific features with properties that PostgreSQL exposes in the `parsed_plans`. Once all the plans are converted and the feature vectors are created, we can train the model.

3.3.1 Mapping Parsed PostgreSQL Plans to the T3 Plan Format

We start from one element, i. e., one plan, of the `parsed_plans` array in the benchmark JSON files. An object in the array is a rooted tree: every node carries `plan_parameters` (operator name, estimated and actual cardinalities, tuple width, planner costs, optional filter predicates, timings, and further fields) and a list `children` of child subtrees.

In `zeroshot_to_t3.py`, we convert this tree recursively into the same nested operator structure that the original T3 implementation expects for Umbra plans [**t3**]. PostgreSQL operator names are mapped to abstract operators (`tablescan`, `join`, `sort`, and `groupby` nodes). The PostgreSQL-specific temporary operators `hash` and `materialize` are mapped to the abstract operator `temp`.

Regarding joins, the child order is normalized so that it matches T3's pipeline rules: i. e., the PG-build subtree becomes the *left* child and the PG-probe subtree the *right* child.

Since the feature vector uses the PostgreSQL-native features and operators of the `parsed_plans`, we need to keep the original, not converted `plan_parameters` available for the feature extraction. Hence, the converted abstract operator names are only used

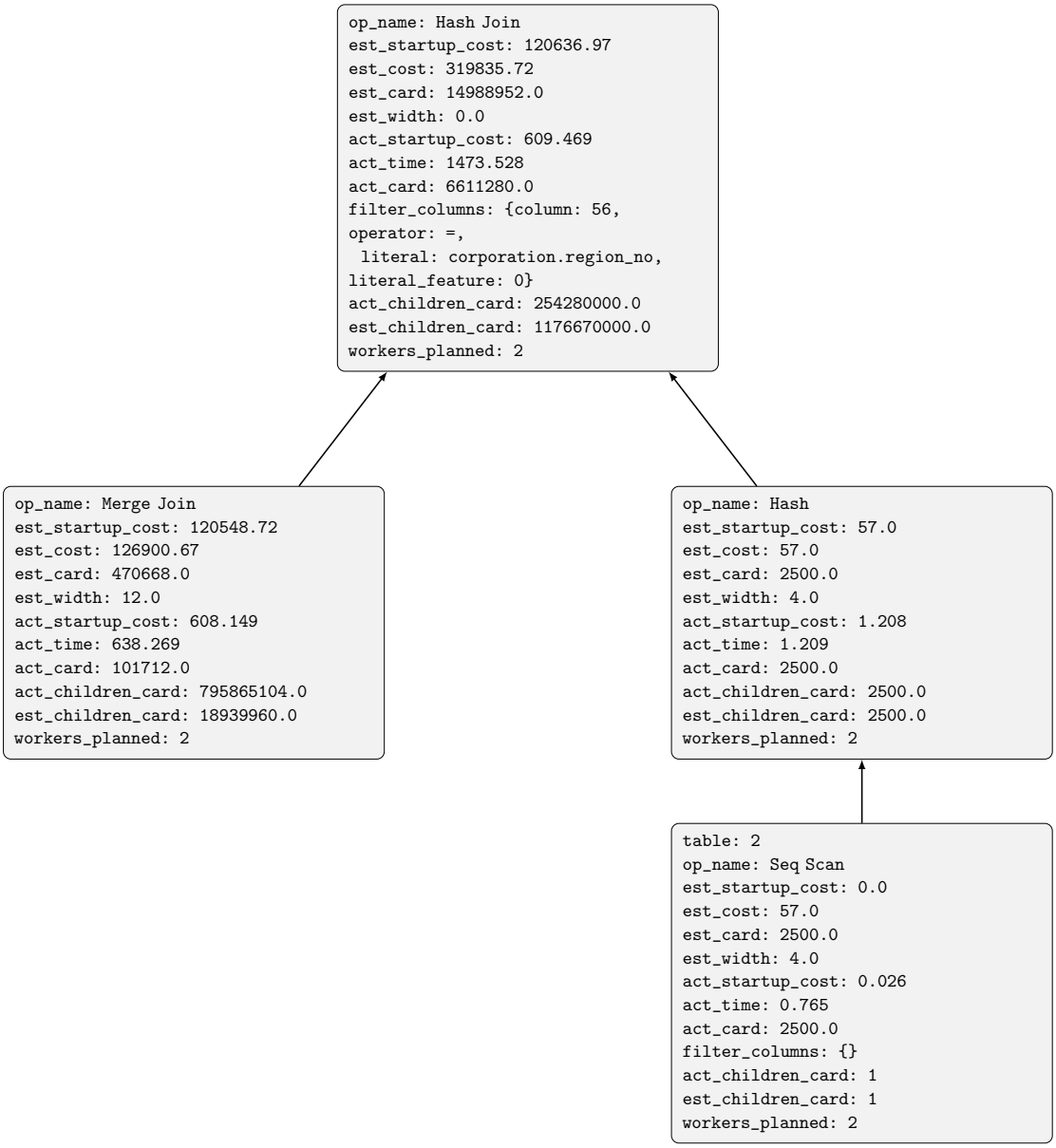

Figure 3.1: Top-node tree cutout of one `parsed_plan` with a `Hash Join` from `credit/workload_100k_s1_c8220`. Child relationships are shown by arrows, and each node box contains selected `plan_parameters` of that node for the displayed cutout.

to match the internal T3 representation for pipeline assignment, but not for the feature extraction.

Consequently, every converted node additionally stores a copy of the original `plan_parameters` under a dedicated `pg` payload. Training and inference, that use PostgreSQL-native features, read from this payload and from the pipeline layout, which will be described in the next sections.

T3 also requires database schema information. However, the Zero-Shot data does not provide this information, so a minimal placeholder schema is used instead.

3.3.2 Pipeline Decomposition

Umbra plans group operators into *pipelines*: sets of operators that can produce tuples in a pipelined fashion, separated by *pipeline breakers*. Namely these are operators/stages, where data is materialized. For instance, sorting, grouping, or materialization, but also the build side of a hash join [**t3**].

Since PostgreSQL plans do not provide such a pipeline breakdown, we need to derive it from the converted tree. We assign each node to a pipeline by walking the converted tree with the same high-level rules as in the T3 work. Thus, PG's sort and aggregate nodes break pipelines; a PG-materialize node is treated as a breaker. Joins place the build side in one pipeline and keep the join together with the probe side in the other pipeline. The pipeline assignment is stored in the `analyzePlanPipelines` list of the converted plan.

For every pipeline, we derive start and stop times. They stem from the per-node actual timings in `plan_parameters` and are accumulated, so that pipeline durations are available for further usage during training.

3.3.3 PostgreSQL-Native Feature Vectors

The original T3 feature mapper for Umbra builds a feature vector from execution phases within each pipeline (e.g. scan, build, probe phases) and from cardinalities and tuple sizes attached to those phases [**t3**]. As PostgreSQL plans contain different data, we use `pg_features.py` to construct the feature vector for each pipeline after the aforementioned pipeline decomposition. It becomes clear, why the `plan_parameters` are kept in the converted tree (`pg` payload entry), as we use them for the feature extraction.

Once the feature vector is constructed, we can train the model. Our feature vector differs from the original T3 feature vector in the following ways:

1. **PostgreSQL-native operator features.** The `parsed_plans` expose PostgreSQL-native operator nodes. Thus, we add entries for these PostgreSQL-specific op-

erators in the feature vector. As no execution stages are available, we aggregate features only per operator. Hence, we do not include the stage-based features of the original T3 feature mapper. This avoids inventing synthetic stages for every node.

2. **PostgreSQL planner cost estimates.** We include sums and extrema of PostgreSQL's `est_cost` and `est_startup_cost` from `plan_parameters` in the feature vector. These are optimizer-internal estimates of work and startup cost and are an additional PostgreSQL-specific signal that is not available in the Umbra feature vector. Consequently, we can include this information in the feature vector.

3. **Planned parallelism.** Another PostgreSQL-specific signal is the planned parallelism. So, we take the sum of `workers_planned` over all operators in the pipeline. Hereby, we include a parallelism signal in our feature vector.

4. **Sizes.** T3 uses tuple sizes in its feature vector. This information is provided by the Umbra plans. However, the parsed PostgreSQL do not contain a perfect equivalent. Instead, they report a single width estimate per node (`est_width`). Thus, an average of `est_width` over all operators in the pipeline is used to represent sizes in the feature vector, replacing the tuple size features of T3.

In the Umbra feature vector of T3, rows are indexed by both operator type and execution phase (scan, build, probe, ...) like `HashJoin_Probe...` [**t3**]. Now with the PostgreSQL implementation, we miss out on the execution phase dimension.

Whereas not an equivalent substitute, we replace this information with a pipeline-level dimension and keep the operator-level features for the individual operators. The pipeline-level block produces additional metadata in a compact manner, e. g., by using averages and sums over operators. For instance, we consider planned parallelism, width, and cost estimates per pipeline and not per operator. Subsequently, the feature vector stays more compact.

Each pipeline's feature vector consists of two concatenated blocks, as Figure 3.2 shows:

1. **Pipeline-level aggregates.** From the available `pg` payload, we try to extract as distinct information as possible.

   Therefore, we start by summarizing different information categories per pipeline with sum, max, and average statistics. First, the `pg_act_card_...` fields capture output cardinality summaries.

   Insights about PostgreSQL-specific cost, parallelism, and width features are captured in the `pg_est_cost_...`, `pg_est_startup_...`, `pg_workers_planned_sum` and `pg_est_width_avg` fields.

Further, the `pg_num_(scan/join/sort/etc.)` family counts operator classes.

Moreover, the `pg_scan_has_filter` and `pg_filter_(and/or/compare/like/in)` counters enumerate scan predicate attributes.

Finally, pipeline-context features describe structure using `pg_pipeline_num_ops` for instance.

2. **Operator-level block.** For every operator that occurs in `parsed_plans`, we assign a slice in the feature vector. If multiple operators of the same type occur in the pipeline, we sum the features over all occurences. In figure 3.2, the excerpt of the operator-type block for the `Nested_Loop join` operator is depicted.

   Depending on the operator type, we include different features. This feature set is derived from T3's Umbra implementation [**t3**]. For example, a join would include different features than a scan, like in the Umbra/T3 approach.

   Regarding the calculations of the features, we follow the Umbra/T3 implementation where adequate. A deviation would be, e. g, how expression percentages are formed due to the different nature of PostgreSQL's filter predicates (CITE).

   Finally, we include the operator count "`const`", which exists for every operator type in both our and the Umbra/T3 implementation [**t3**].

In total, the feature vector is fixed and has 154 dimensions (24 pipeline-level features and 130 operator-type features). Empty features are zeroed to keep the feature vector dimension.

Regarding the cardinalities, either the actual or the estimated cardinalities are used in the feature vector, depending on which is selected in training. However, even if estimates are used, the row names are still called "actual" cardinalities. This caveat is a design decision to keep the row names completely uniform.

Once the feature vectors for all pipelines are constructed, we get the feature matrix for the whole plan. We proceed to train the model in the next section.

3.4 Training

Concerning the training, we keep T3's LightGBM decision tree setup [**t3**]. Then, we use the newly constructed PostgreSQL-native feature matrix. A main training difference is that we train directly on pipeline durations as labels rather than using tuple-based targets.

Pipeline-level block (excerpt; first 12 of 24 features)	
`pg_act_card_sum`	6541.0
`pg_act_card_max`	6537.0
`pg_est_cost_sum`	16882.95
`pg_est_cost_max`	16877.3
`pg_est_startup_sum`	125.82
`pg_workers_planned_sum`	4.0
`pg_est_width_avg`	10.0
`pg_num_scan`	1.0
`pg_num_join`	1.0
`pg_scan_has_filter`	0.0
`pg_filter_compare_count`	0.0
`pg_pipeline_num_ops`	2.0
Operator-type block (excerpt; Nested_Loop slice; 130 entries in full)	
`Nested_Loop_const`	1.0
`Nested_Loop_in_percentage`	0.11473152822395594
`Nested_Loop_right_card`	98378.0
`Nested_Loop_out_percentage`	0.008260670032124828
(remaining operator-type entries omitted)	

Figure 3.2: Excerpts of the pipeline-level block (first twelve features) and of the `Nested_Loop` operator entries in the operator-type block. They are from one pipeline of query `index_workload_100k_s2_c8220_890` on the TPC-H database. The full vector has 154 dimensions; remaining pipeline and operator entries are omitted.

3.4.1 Training Targets and Model Configuration

Concerning the training labels, we use the observed pipeline duration. In T3 [**t3**], the training target is the per-tuple-time instead, which is the pipeline duration divided by a pipeline scan cardinality. We will compare the per-tuple-time approach with the pipeline duration approach in the REFERENCEABLATIONSTUDY.

Regarding inference, our predicted query runtime is the sum of the predicted pipeline durations.

Our model uses gradient boosted decision trees as implemented in LightGBM [**lightgbm**]. The booster is trained with a mean absolute percentage error (MAPE) objective on the training rows. This is in line with the original T3 work [**t3**].

Furthermore, we use the same training parameters as in the original T3 work [**t3**]. Hence, the pipeline duration targets t are transformed as $-\log(t)$ to stabilize training. Also following T3, we cap the number of boosting rounds at 200 trees to mitigate overfitting.

3.4.2 Data Splits and Reproducibility

Generalization. We match the training intention of the Zero-Shot benchmark [**zeroshot**] and of T3's cross-instance evaluation [**t3**]. Both aim to measure whether the trained model transfers to an unseen database instance. Thus, we use a leave-one-out split on the `parsed_plan` files: We specify one instance as a holdout and use its `parsed_plan` files as the test set. All other instances form the training data.

Concretely, `training_zeroshot_tpch_holdout.py` uses the `--holdout` argument to specify the holdout instance (default: `tpc_h`). Then, we perform a training for all test instances with `run_all_holdouts.py`.

Train and Validation Splits. Mirroring T3's train/validation split, we sample 80% of the training data for training and 20% for validation [**t3**]. The default seed used for shuffling the training data is 42.

Environment. Model training and evaluation for the results reported in this thesis were run on a personal laptop: an Apple MacBook Pro (16-inch, 2021) with the Apple M1 Pro processor (10 CPU cores, 16 GPU cores), 16 GB memory, and a 512 GB SSD, under macOS 15.7.3.

3.5 Other Performance Prediction Implementations

Add other implementations that you want to compare to in evaluation.

3.6 Section

Citation test [**latex**].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

3.6.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 3.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

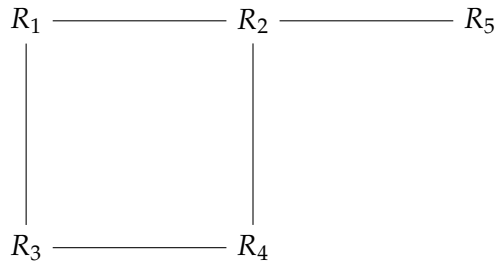

Figure 3.3: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

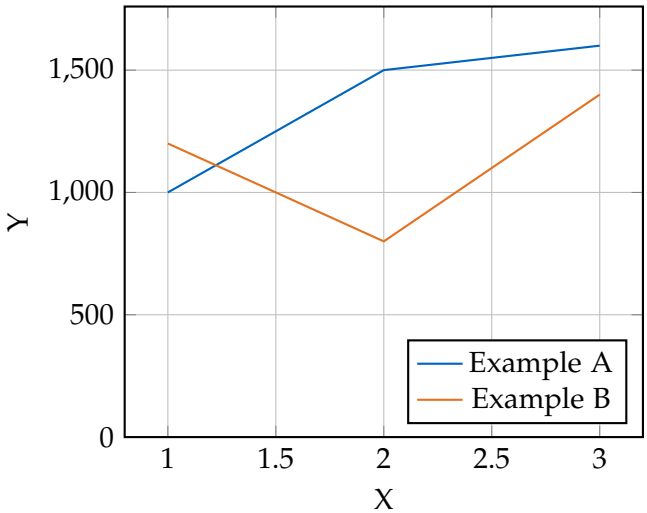

Figure 3.4: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 3.5: An example for a source code listing.

4 Evaluation

4.1 Ablation Study

4.1.1 PostgreSQL-native Features vs. Direct Transfer

4.1.2 Actual vs. Estimated Cardinalities

4.1.3 Pipeline vs Tuple targets

4.2 Section

Citation test [**latex**].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

4.2.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 4.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

[1] `https://ctan.org/pkg/acronym`

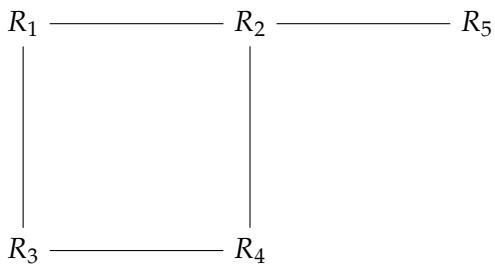

Figure 4.1: An example for a simple drawing.

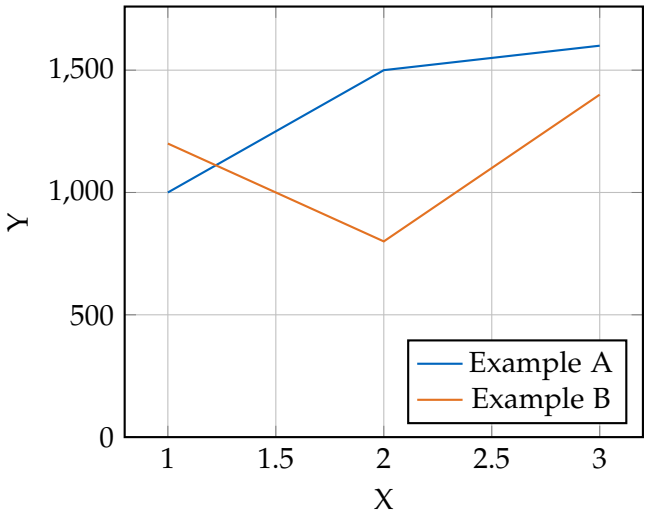

Figure 4.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 4.3: An example for a source code listing.

5 Future Work

Data from Zeroshot kann limitierender faktor sein. INPUT CARD immer 1 somit impact auf feature berechnung.. JOB nur 77 von 113 in zeroshot.

Pipeline-level aufbrechen auf operators wie z. B. planned parallelism, etc..., da wir dann bisschen vom pipeline level wegkommen vielleicht per-tuple-target nochmal probieren.

Versuchen auch mal weg von pipeline level zu gehen und feature vector auf per query level zu machen..

5.1 Section

Citation test [**latex**].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

5.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 5.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

[1] `https://ctan.org/pkg/acronym`

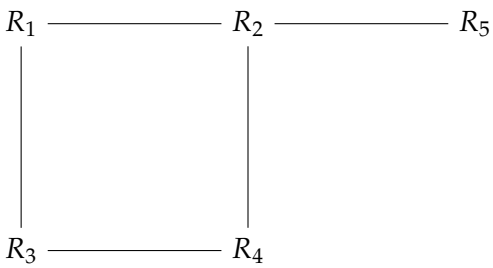

Figure 5.1: An example for a simple drawing.

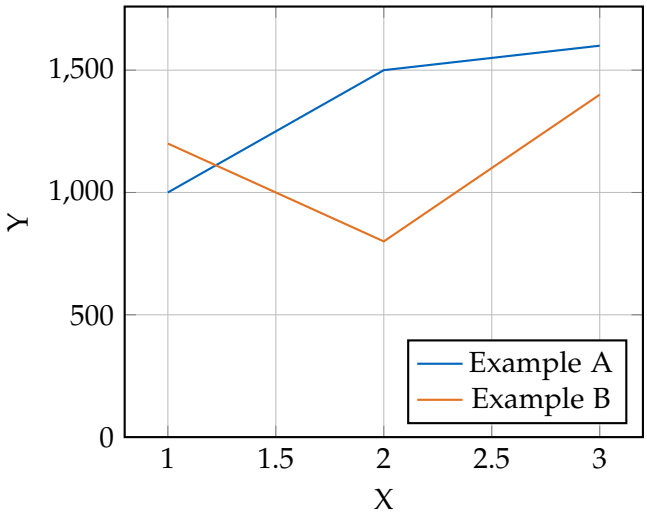

Figure 5.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 5.3: An example for a source code listing.

6 Conclusion

6.1 Section

Citation test [**latex**].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

6.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 6.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

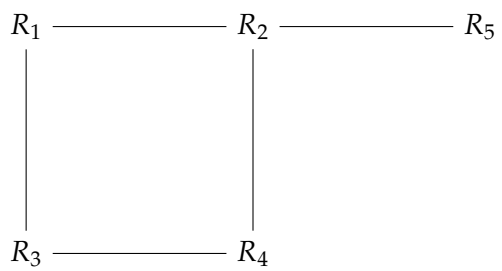

Figure 6.1: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

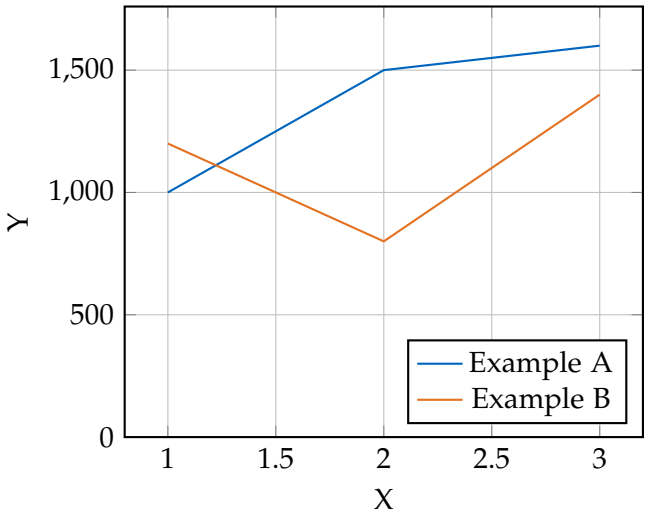

Figure 6.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 6.3: An example for a source code listing.

Abbreviations

TUM Technical University of Munich

List of Figures

1.1 Example drawing 1
1.2 Example plot 2
1.3 Example listing 2

2.1 Example drawing 9
2.2 Example plot 9
2.3 Example listing 9

3.1 Top-node cutout with Hash Join 12
3.2 Excerpt of a PostgreSQL-native pipeline feature vector 16
3.3 Example drawing 18
3.4 Example plot 19
3.5 Example listing 19

4.1 Example drawing 21
4.2 Example plot 21
4.3 Example listing 21

5.1 Example drawing 23
5.2 Example plot 23
5.3 Example listing 23

6.1 Example drawing 24
6.2 Example plot 25
6.3 Example listing 25

List of Tables

1.1 Example table 1

2.1 Example table 8

3.1 Example table 18

4.1 Example table 20

5.1 Example table 22

6.1 Example table 24